Day 14 Task :
MCP & Self-Host n8n Setup

November 17, 2025

1 Introduction

This report outlines the critical steps for setting up a self-hosted n8n instance and integrating it with Model Context Protocol (MCP) servers. The primary advantage of this setup is to enable the use of n8n Community Nodes, specifically the MCP nodes, which are not available on the standard n8n cloud service.

2 Why Self-Host n8n?

The main driver for self-hosting is to gain access to **Community Nodes**. These are custom nodes developed by the n8n community that provide extended functionalities. The `n8n-nodes-mcp` package is a community node and is required to connect n8n to MCP servers.

3 Self-Hosting n8n with Elestio

The video uses Elestio as a hosting provider because it simplifies the deployment process for non-developers, managing the installation, configuration, and security without requiring terminal commands.

3.1 Elestio Setup Process

1. **Account Creation:** Sign up for a free trial on the Elestio website.
2. **Billing:** Add a payment method and add credits to the account. Account activation may take a few minutes.
3. **Create n8n Service:**
   - In the Elestio dashboard, click ”Add a new service” and search for ”n8n”.
   - Select a cloud provider (e.g., Hetzner) and a service plan. (Note: MCP servers can be resource-intensive, so a ”Medium” plan or higher is recommended over the smallest option).
   - Assign a name to the service, which will become part of its unique domain.
   - Click ”Create Service” and wait for the instance to deploy.
4. **Access Instance:** Once deployed, Elestio provides the URL for your new n8n instance. You must complete the n8n ”owner account” setup on your first visit.

4 MCP (Model Context Protocol) Setup

Once your self-hosted n8n instance is running, you must install and configure the MCP nodes.

4.1 Step 1: Install the MCP Community Node

1. In your self-hosted n8n instance, navigate to **Settings** → **Community Nodes**.
2. In the ”Install” field, enter the package name: `n8n-nodes-mcp`
3. Click ”Install”. This will add the ”MCP Client” node to your n8n palette.

4.2 Step 2: Enable MCP for AI Agents (Critical)

By default, community nodes are not allowed to be used as tools by an AI Agent for security reasons. This must be enabled in your hosting environment.

1. Go back to your **Elestio dashboard**.
2. Navigate to your n8n service and select **Software** → **Update Config**.
3. Scroll down to the **Environment** variables section.
4. Add the following new line:
   `N8N_COMMUNITY_PACKAGES_ALLOW_TOOL_USAGE: true`
5. Click **Update and Restart** to apply the changes.

After the service restarts, your n8n AI Agent will be able to see and use the MCP Client node as a tool.

4.3 Step 3: Connect to an MCP Server

The ”MCP Client” node in n8n needs to connect to an MCP server to function. Each server (e.g., for Airbnb, Brave Search, AirTable) requires its own credential.

1. Add the **MCP Client** node to your n8n workflow.
2. In the ”Credential” field, select ”Create New”.
3. Find the documentation for the MCP server you want to use (e.g., from the ‘mcp-servers‘ GitHub repository).
4. Configure the credential based on the server’s documentation. This typically involves three fields:
   - **Command:** The command to run the server (e.g., `npx`).
   - **Arguments:** The specific package and flags for the server (e.g., `-y @brave/mcp-server`).
   - **Environment Variables:** (If required) This is where you add API keys, formatted as `KEY=VALUE` (e.g., `BRAVE_API_KEY=sk-123...`).
5. Save the credential.

4.4 Step 4: Using the MCP Node

The MCP Client node has several operations. The two most important are:

- **List Available Tools:** This operation connects to the server and returns a list of all available functions (e.g., `airbnb : search`, `brave : web_search`), their descriptions, and the data schema they require. This is useful for testing the connection and for prompting.
- **Execute Tool:** This is the operation you use to perform an action. This is the node you should connect to an AI Agent as a tool, allowing the agent to dynamically choose which tool to run (`Tool Name`) and what data to send (`Tool Parameters`).

4.5 Limitations Mentioned

- **Complexity:** Some MCP servers are complex. For example, to list records in AirTable, the agent must first find the `Base ID`, then use that ID to find the `Table ID`, before finally listing the records. This multi-step logic may require detailed prompting.
- **Stability:** Not all servers listed in community repositories may be stable or functional. The video notes that an attempt to connect to the Perplexity MCP server failed.